GraphHunter

Arquitectura

Lucas Sotomayor
Base4Security

13 de mayo de 2026

Índice general

1. Introducción **1**
- 1.1. Audiencia . . . 1
- 1.2. Alcance . . . 1
- 1.3. Mapa del libro . . . 1
- 1.4. Convenciones . . . 2

2. Capas de cizalla **3**
- 2.1. La metáfora . . . 3
- 2.2. Tres capas . . . 3
- 2.3. Criterios de clasificación . . . 3
- 2.4. Diferencia con SOLID uniforme . . . 4
- 2.5. Cómo se refleja en el árbol . . . 4
- 2.6. Lo que ganamos . . . 5

3. Grafo de dependencias **6**
- 3.1. Catorce crates Rust más dos vecinos . . . 6
- 3.2. Dependencias ideales . . . 6
- 3.3. Dependencias reales . . . 7
- 3.4. Ausencia de ciclos . . . 7
- 3.5. Diagrama resumen . . . 7

4. Capa de núcleo **8**
- 4.1. `core/graph-engine` . . . 8
- 4.2. `core/matcher-ffi` . . . 9
- 4.3. `core/gnn` . . . 9
- 4.4. Invariantes que defiende el núcleo . . . 9

5. Capa de plataforma **11**
- 5.1. `platform/api` — la fachada única . . . 11
- 5.2. `platform/dsl` . . . 11
- 5.3. `platform/parsers` . . . 12
- 5.4. `platform/sources` . . . 12
- 5.5. `platform/canonical` . . . 12
- 5.6. `platform/vrl` . . . 12
- 5.7. `platform/constrained-decode` . . . 12
- 5.8. `platform/siem` . . . 12
- 5.9. `platform/local-llm` . . . 12
- 5.10. `platform/mcp` (TypeScript) . . . 12
- 5.11. Qué tienen en común . . . 13

6. Capa de aplicaciones **14**
6.1. `apps/tauri` . . . 14
6.2. `apps/cli` . . . 14
6.3. `platform/mcp` . . . 15
6.4. Por qué el MCP es plataforma, no aplicación . . . 15
6.5. Convenciones de aplicaciones . . . 15

7. Patrones transversales **16**
7.1. Typestate . . . 16
7.2. RAII sobre handles C . . . 16
7.3. Canonical API . . . 17
7.4. Model Gate . . . 17
7.5. Registry de extensiones . . . 17
7.6. Por qué estos cinco y no otros . . . 18

8. Decisiones arquitectónicas **19**
8.1. ADR-001 — Canonical API Layer . . . 19
8.2. ADR-002 — FFI Isolation (cerrada el 2026-05-07) . . . 19
8.3. ADR-003 — GNN v2 Experimental . . . 19
8.4. ADR-004 — MCP Tools Architecture . . . 20
8.5. ADR-005 — DSL Extensibility . . . 20
8.6. ADR-006 — WIP Absorption . . . 20
8.7. ADR-007 — Ingestor agéntico en tres carriles . . . 20
8.8. Cómo encadenan . . . 21

9. Deudas y trabajo futuro **22**
9.1. Dos aristas `core` → `platform` (con *guardrail* automatizado) . . . 22
9.2. Tests flaky conocidos . . . 22
9.3. GNN v2 sin modelo entrenado . . . 22
9.4. Follow-ups agentic slow lane . . . 23
9.5. Deudas cerradas en el ciclo 2026-04 . . . 23
9.6. Formato del libro . . . 24

10. Ingestor agéntico (tres carriles) **25**
10.1. Motivación . . . 25
10.2. Flujo end-to-end . . . 26
10.3. Fast lane: VRL compilado . . . 26
10.4. Slow lane: LLM-as-compiler offline . . . 26
10.5. Learning lane: drift-as-signal . . . 27
10.6. Estabilidad de cada carril . . . 27
10.7. Relación con el resto de la arquitectura . . . 28
10.8. Qué queda fuera de este libro . . . 28

11. Estado de implementación **29**
11.1. Matriz *stable / experimental / future* . . . 29
11.2. Invariantes del *default build* . . . 29
11.3. Puntos de entrada para auditores . . . 30
11.4. Política de cambios al capítulo . . . 30

A. ADRs: registros de decisión **31**

B. Métricas del código **34**
B.1. Líneas por crate 34
B.2. Tests por crate 35
B.3. Propiedades formales con property-testing 35
B.4. Ciclos arquitectónicos 35
B.5. Aristas entre capas 35
B.6. Benchmarks definidos 36
B.7. Cambios respecto al pre-rediseño 36

Capítulo 1

Introducción

GraphHunter es un motor de caza de amenazas sobre grafos temporales construidos a partir de logs de seguridad. Este libro describe *cómo está construido*: qué capas lo componen, qué piezas viven en cada capa, por qué se eligieron esos límites y qué patrones se repiten en todo el código.

Véase también el libro *Algoritmos* (`docs/algoritmos/algoritmos.pdf`) para la guía pedagógica del diseño algorítmico: pipeline de ingesta, matcher tier-por-tier (DFS / LFTJ-K4 / Yannakakis chain / Yannakakis tree-walk), índices auxiliares (NLF, k-hop reachability), DSL, y el plano de obra de las stages futuras L3. El apéndice E del mismo libro consolida el *mapa interno* del motor (referencia técnica para quien va a tocar el código).

1.1. Audiencia

Está pensado para tres lectores distintos:

- **Nuevo contribuidor**: necesita el mapa del repositorio y el razonamiento detrás de la estructura antes de abrir su primer pull request.
- **Revisor arquitectónico**: evalúa propuestas de cambio que tocan múltiples capas, o sanciona nuevas dependencias entre crates.
- **Operador técnico**: opera el sistema en producción y necesita entender los límites de aislamiento (canonical API, model gate) para diagnosticar problemas.

1.2. Alcance

Este libro cubre *la arquitectura en sí*: estructura, decisiones, patrones, deudas. No cubre:

- Las **superficies externas** (HTTP API, MCP tools, DSL, API pública del matcher Rust) — eso vive en el libro de Especificación.
- El **flujo de uso** del sistema (ingestar, cazar, anotar, exportar) — eso vive en el Manual.
- La **justificación experimental** de algoritmos (matcher SIMD, DFS temporal, scoring compuesto) — eso vive en el Paper.

1.3. Mapa del libro

- El capítulo 2 introduce el modelo de capas de cizalla (*shearing layers*) que organiza el proyecto.
- El capítulo 3 muestra el grafo de dependencias inter-crate y las propiedades que garantiza.

- Los capítulos 4, 5 y 6 recorren cada capa en detalle.
- El capítulo 7 describe patrones transversales: typestate, RAII, canonical API, model gate.
- El capítulo 8 resume las seis decisiones arquitectónicas (ADR-001 a ADR-006) que fundamentan el estado actual.
- El capítulo 9 documenta honestamente la deuda conocida y el trabajo futuro.
- El apéndice A contiene las ADRs textualmente. El apéndice B reporta métricas cuantitativas (LOC, tests, crates, ciclos).

1.4. Convenciones

- `rutas/de/archivo` en tipografía monoespaciada.
- `identificadores_de_código` igualmente monoespaciados.
- Crates Rust se escriben con su path en el repositorio: `core/graph-engine` significa "el crate que vive en `core/graph-engine/Cargo.toml`".
- Cuando un párrafo dice "ADR-00N" se refiere al apéndice A, que contiene el documento completo.

Capítulo 2

Capas de cizalla

2.1. La metáfora

Stewart Brand, en *How Buildings Learn*, observa que un edificio no cambia a la misma velocidad en todas sus partes: los cimientos duran décadas, la estructura medio siglo, la piel del edificio treinta años, los servicios (plomería, electricidad) una década, la distribución interior unos cinco años, y las cosas que ponemos encima de las mesas cambian a diario. Cada capa cede (*shears*) a distinta velocidad; si se las acopla rígidamente, el ritmo rápido arrastra al lento y se termina rompiendo.

GraphHunter aplica la misma idea al software. Hay piezas que cambian poco (algoritmos de grafo, intersección AVX2 en `simd\_rust.rs`, formato canonical OCSF) y piezas que cambian seguido (un nuevo parser de log, una nueva tool MCP, una pantalla nueva del Tauri). Querer aplicar exactamente los mismos estándares de rigor a ambas mata la velocidad; querer aplicar los mismos estándares de flexibilidad mata la corrección.

2.2. Tres capas

El repositorio se organiza en tres capas:

Núcleo (`core/`) — Algoritmos y tipos fundamentales. Contratos matemáticos explícitos, property-based testing, benchmarks. Abstracciones sólo cuando hay variación real. Cambia lento; cada cambio pesa.

Plataforma (`platform/`) — Superficies de extensión: API, DSL, parsers, fuentes de log, canonicalización, MCP. SOLID clásico aplica aquí: OCP e ISP, traits como contratos públicos. Cambia a ritmo medio; lo que se agrega debe mantener compatibilidad.

Aplicaciones (`apps/`, `platform/mcp/`) — Piel visible: Tauri (GUI), CLI, MCP server TypeScript. YAGNI agresivo, código pragmático y directo. Cambia rápido; lo importante es que el cambio sea obvio.

La regla de acoplamiento entre capas es **unidireccional hacia abajo**: `apps/` puede depender de `platform/`, y `platform/` puede depender de `core/`. Lo inverso no se permite. Esto garantiza que un cambio en una pantalla del Tauri nunca force un cambio en el algoritmo de matching, y que un nuevo parser no pueda corromper el núcleo algorítmico.

2.3. Criterios de clasificación

La decisión de qué va en qué capa no es una votación. Se aplica, en orden, este checklist:

1. ¿El módulo implementa un **algoritmo con contrato matemático** (DFS, intersección, scoring)? Núcleo.

2. ¿El módulo define un **trait de extensión** que terceros (o futuros módulos) van a implementar (`LogParser`, `DslExtension`, `Tool`)? Plataforma.

3. ¿El módulo es una **superficie pública versionable** (endpoints HTTP, tools MCP, DTOs, C ABI, DSL)? Plataforma.

4. ¿El módulo es **pass-through** o binding 1:1 a una pieza ya existente (Tauri command que reenvía a la API, botón que llama a un endpoint)? Aplicación.

5. ¿El módulo es **setup** de proceso (main, argparse, router de HTTP, IPC bridge)? Aplicación.

Cuando un módulo cumple varios criterios, el que aparezca primero gana (algoritmo > trait > superficie > pass-through > setup).

2.4. Diferencia con SOLID uniforme

Un enfoque SOLID uniforme trataría a todas las piezas del código con la misma vara: interfaces para todo, inversión de dependencias para todo, apertura/cerrada para todo. El costo es real: traits con una sola implementación que sólo existen para hacer "tests unitarios" con mocks, arquitecturas hexagonales con tres niveles de indirección entre un HTTP handler y una query SQL, y una cognitive load desproporcionada para cambios triviales.

Shearing layers no niega SOLID; lo *localiza*. SOLID reina en `platform/`, donde la extensibilidad es una feature de primera clase. En `core/` se prefiere tipos fuertes, invariantes codificadas y property tests; las abstracciones nacen sólo cuando hay una segunda implementación real. En `apps/` se prefiere código imperativo plano, legible sin saltos de archivo.

2.5. Cómo se refleja en el árbol

```
GraphHunter/
|-- core/
|   |-- graph-engine/       # algoritmos de grafo, DFS, anomaly,
|   |                       #   simd_rust.rs (AVX2 puro Rust)
|   |-- matcher-ffi/        # thin Rust forwarder (post-2026-05-07)
|   |-- gnn/                # scorer ONNX (v1 estable, v2 gate)
|   `-- gnn/src/v2/         # experimental, feature-gated
|-- platform/
|   |-- api/                # GraphHunterApi (facade)
|   |-- canonical/          # OCSF v1.4 projection
|   |-- constrained-decode/ # guardrail LLM
|   |-- dsl/                # hypothesis DSL
|   |-- local-llm/          # trait LocalLlm + backends
|   |-- parsers/            # trait LogParser + 7 impl
|   |-- siem/               # Sentinel/Elastic streaming
|   |-- sources/            # trait LogSource
|   |-- vrl/                # FieldConfig -> VRL compiler
|   `-- mcp/                # TypeScript MCP server (54 tools)
|-- apps/
|   |-- tauri/              # GUI Tauri + React
|   `-- cli/                # CLI (usa platform/api in-proc)
`-- legacy/                 # gateway Go, archivado
```

2.6. Lo que ganamos

Cuatro propiedades se siguen directamente de esta estructura:

- **Cambios aislados**: reemplazar un parser no afecta al núcleo; cambiar el scoring no afecta al DSL; reorganizar la GUI no afecta a nada del backend.
- **Revisión proporcional**: un PR que toca sólo `apps/` pasa con revisión ligera; uno que toca `core/` exige benchmarks y pruebas de propiedad; uno que toca `platform/` exige compatibilidad de API.
- **Tests apropiados**: property-based y equivalencia en el núcleo, contractuales en la plataforma, smoke en aplicaciones. Sin uniformar a lo más costoso.
- **Onboarding escalonado**: un contribuidor nuevo empieza tocando `apps/`, gana contexto, después migra a `platform/`, y sólo entonces toca `core/`.

El resto del libro asume este mapa. El capítulo siguiente muestra el grafo de dependencias que lo hace verificable.

Capítulo 3

Grafo de dependencias

3.1. Catorce crates Rust más dos vecinos

El repositorio contiene 14 crates Rust y un paquete TypeScript (`platform/mcp/`). El cuadro 3.1 los enumera por capa. *Nota 2026-05-07*: hasta esa fecha existía además un crate C++ vecino (`libgraphmatch/`) con el matcher SIMD; fue eliminado tras la migración al DFS puro-Rust.

Capa	Crate/paquete	Rol
core	`core/graph-engine`	grafo, DFS, anomaly, interner
core	`core/matcher-ffi`	thin forwarder Rust (post-2026-05-07)
core	`core/gnn`	scorer ONNX (v1 estable, v2 gate)
platform	`platform/api`	`GraphHunterApi` facade
platform	`platform/canonical`	proyección OCSF v1.4
platform	`platform/constrained-decode`	guardrail LLM
platform	`platform/dsl`	gramática de hipótesis
platform	`platform/local-llm`	backends mock/candle
platform	`platform/parsers`	`LogParser` + impls
platform	`platform/siem`	Sentinel / Elastic streaming
platform	`platform/sources`	`LogSource` + conectores
platform	`platform/vrl`	compilador FieldConfig → VRL
apps	`apps/tauri/src-tauri`	GUI Tauri + commands
apps	`apps/cli`	CLI
vecino TS	`platform/mcp/`	54 tools MCP

Tabla 3.1: Crates y paquetes por capa.

3.2. Dependencias ideales

La regla enunciada en el capítulo anterior, aplicada al grafo, produce tres aristas de capa:

- `apps/*` → `platform/*`
- `platform/*` → `core/*`

Nota 2026-05-07: hasta esa fecha existía una tercera arista `core/matcher-ffi` → `libgraphmatch/` a través de `build.rs` y un bloque `extern "C"`. Tras la eliminación del C++ tree esa arista ya no existe; el repositorio es 100 % Rust+TS.

El vecino TypeScript `platform/mcp/` no depende de crates Rust en tiempo de compilación: cada tool llama al HTTP API que expone el Tauri (o un binario dedicado). Lo clasificamos como *plataforma* por rol, aunque técnicamente es su propio proceso.

3.3. Dependencias reales

Dos aristas rompen la regla hacia abajo en el estado actual:

- `core/graph-engine` → `platform/sources` (referencia por conveniencia al tipo `AadEventBatch` usado por el GNN bridge).
- `core/graph-engine` → `platform/dsl` (acoplamiento pragmático introducido al extraer el DSL; el parser necesita helpers del core y el core necesita tipos del DSL para materializar hipótesis).

Son dos aristas de *core a platform*, dirección prohibida por el modelo. No son ciclos (el grafo sigue siendo un DAG), pero violan el ideal de shearing. El capítulo 9 discute cómo eliminarlas en el siguiente ciclo de trabajo.

3.4. Ausencia de ciclos

La verificación se hace fuera de banda con:

```
cargo metadata --format-version 1 | \
  jq -r '.packages[] | "\(.name) \(.dependencies[].name)"' | \
  ...construcción de DAG y detección de ciclos
```

El resultado al cierre de la Fase 4 del rediseño: **cero ciclos**. Dos edges "indeseados" documentados, pero nada bidireccional. Es una propiedad que se reverifica en cada PR que toca `Cargo.toml`.

3.5. Diagrama resumen

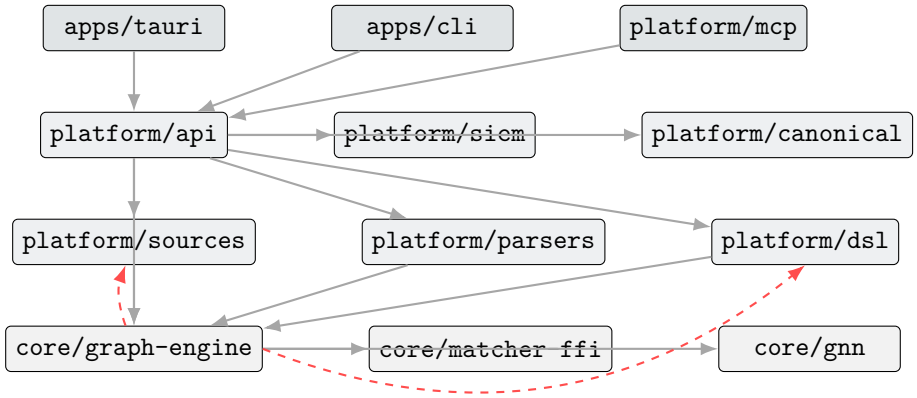


Figura 3.1: Grafo de dependencias (simplificado). Las flechas rojas punteadas son las dos edges "core → platform" que rompen la regla; capítulo 9 las discute.

El grafo es un DAG; la única bidireccionalidad conceptual (`core` ↔ `platform`) es la deuda explícita que se quiere eliminar.

Capítulo 4

Capa de núcleo

`core/` contiene tres crates: `graph-engine`, `matcher-ffi` y `gnn`. Juntos definen el motor algorítmico del sistema.

4.1. `core/graph-engine`

Es el crate más grande del repositorio. Agrupa:

- **Modelo de grafo**: multigrafo temporal dirigido sobre entidades (`Entity`) y relaciones (`Relation`). Un nodo es una tupla `(type, value)` — `(IP, "10.0.0.3")` — y una arista es una tupla `(source, type, target, timestamp, attrs)`. La conservación del tiempo es una invariante: toda arista tiene un `t` y los algoritmos se apoyan en ordenamiento monotónico.
- **DFS temporal guiado por score**: iterativo (sin recursión), poda por score parcial. Es la ruta canónica post-2026-05-07 (cuando `libgraphmatch` fue eliminado tras demostrarse 3-25× más lento que este DFS). La intersección AVX2 sorted-set sobrevive en `core/graph-engine/src/simd\_rust` con runtime dispatch.
- **Anomaly scoring**: cinco componentes formalmente definidos en `scoring/components.rs`. Cada uno produce un valor en $[0, 1]$ y la composición se hace con pesos normalizados. Ver el libro Paper para fórmulas.
- **Interner**: deduplicación de strings compartidos (`SymbolId`) con un único backend.
- **Invariantes de hipótesis**: predicados que validan forma de la hipótesis (treewidth, k-simplicity, shape catalog) antes de ejecutarla.
- **Ingest pipeline**: coverage, cancellation, métricas, enrichment — el plumbing que convierte un flujo de eventos en un grafo consistente.
- **Planner de operadores físicos** (`src/planner.rs`): inspecciona la hipótesis y elige entre el DFS general y *fast-paths* estructurales. Hoy detecta K_4 homogéneo (*6 pasos*, *tipo de entidad y relación constantes*, *sin predicados*) y rutea a `search\_temporal\_pattern\_k4\_lftj`; el *enum* `DispatchPlan` está diseñado para crecer a K_5, C_6 y *shapes* más complejas sin tocar *call-sites*. La detección estructural complementa al detector legacy por *name marker* de S3.
- **Indice de *k-hop label reachability*** (`src/khop\_reach.rs`): *bitmap* 9-bit por vértice indicando qué tipos de entidad son alcanzables en a lo sumo `MAX\_DEPTH=4` saltos, con extensión M2 que guarda mín / máx *first-edge timestamp* por tipo para descartar *starts* cuya ventana temporal no permite alcanzar el destino. Costa $\sim$ 144 bytes/vértice, falla siempre conservadora.

- **Análisis de simetría *nauty*** (`src/nauty\_symmetry.rs`, *cfg(unix) + feature symmetry-nauty*): FFI a `nauty-Traces-sys` para computar Aut(Q) de *pattern graphs* arbitrarios y emitir *partial-order constraints chain* $v_0 < v_1 < \ldots < v_{k-1}$ por órbita (cota Grochow-Kellis, RECOMB 2007). En *Windows* y sin *feature* retorna lista vacía (*soundness-preserving no-op*).
- **LFTJ + *canonical operators*** (`src/lftj.rs`): núcleo *Worst-Case Optimal Join* (Veldhuizen ICDT 2014) sobre `MaterializedCsrTrie`; cuenta y enumera triángulos, K_4, K_5 y C_6 dirigidos. Cada *shape* expone una sibling *canonical* con *symmetry-breaking hardcoded* (cadena para cliques, *star-from-min* para C_6) que elimina el factor |Aut| de redundancia, y una *rayon-parallel* *_par* para los dos modos.

En cifras: aprox. 45 000 líneas de Rust, el conjunto de tests más denso del repositorio (benches de criterion, proptest para equivalencia, proptest para propiedades de anomaly).

4.2. `core/matcher-ffi`

Tras la eliminación de `libgraphmatch` (2026-05-07) este crate quedó reducido a un *thin forwarder* Rust-puro de ∼30 líneas que delega a `GraphHunter::search\_temporal\_pattern\_smart` / `search\_temporal\_pattern`. Se conserva por estabilidad del nombre público; es candidato a ser eliminado cuando los *call-sites* migren al engine directamente.

Estado pre-migración (referencia histórica): el crate era la fachada segura para `libgraphmatch`, único punto del repositorio con un bloque `extern "C"` en Rust. Exponía `GmGraph<Building/Finalized>` (typestate), `GmMatcher`/`GmResults` (RAII) y dos utilidades de chequeo en load-time, `gm\_abi\_version()` y `gm\_simd\_isa()`. ADR-002 cubre las decisiones de aquel diseño.

4.3. `core/gnn`

Scorer basado en inferencia ONNX. Está organizado en subdirectorios según estabilidad:

- `v1/`: v1 estable con `K_MAX = 32`, `D_NODE = 16`, 5 clases. Se compila por defecto.
- `v2/`: v2 experimental detrás de `feature = "gnn-v2-experimental"`. Añade dimensiones para contexto de identidad en la nube. El modelo no está entrenado sobre todos los datasets objetivo.
- `common/model_gate.rs`: *model gate* — un cargador que valida el archivo del modelo antes de habilitar inferencia (checksum, firma, dimensiones esperadas). Si la validación falla, el GNN queda deshabilitado pero el binario sigue corriendo; anomaly score se computa sin el quinto componente.

ADR-003 registra el aislamiento detrás del gate y la razón del feature flag.

4.4. Invariantes que defiende el núcleo

Desde el punto de vista del resto del sistema, el núcleo garantiza:

1. **Grafo bien formado**: nunca hay aristas apuntando a nodos inexistentes, los timestamps son crecientes dentro de una hipótesis, los símbolos están internados una única vez.
2. **Equivalencia AVX2/escalar**: `simd\_rust` despacha a AVX2 cuando el host lo soporta y al merge escalar en otro caso; ambos caminos devuelven el mismo conteo de intersección (15 tests + paridad LFTJ *naive* = *LFTJ* = *LFTJ+SIMD* sobre triángulos).
3. **Scores en** $[0, 1]$: toda componente es normalizada; la composición es convexa.

4. **Sin frontera C/C++ que defender**: tras la eliminación de `libgraphmatch` (2026-05-07) el árbol es 100 % Rust; el único `extern "C"` residual es a `nauty-Traces-sys` bajo `cfg(unix) + feature symmetry-nauty`.

5. **Reproducibilidad**: dado el mismo grafo, la misma hipótesis y el mismo modelo GNN, se produce el mismo ranking.

El núcleo hereda el nombre del libro Brand: es el *structure*. Cambia lento; cuando cambia, exige pruebas formales; y todo el resto del sistema descansa sobre sus invariantes.

Capítulo 5

Capa de plataforma

`platform/` agrupa nueve crates Rust y un paquete TypeScript. Tienen en común ser *superficies versionables*: lo que publican es contrato público visto por consumidores internos o externos.

5.1. `platform/api` — la fachada única

`GraphHunterApi` es el objeto central. Todos los consumidores (Tauri, CLI, MCP, cualquier proceso embedder) obtienen una instancia de esta struct y llaman métodos organizados por dominio:

- `hunt::run_hunt`, `hunt::expand_node`
- `ingest::ingest_events`, `ingest::stream`
- `export::to_ocsf`, `export::to_iocs`
- `session::create`, `session::load`, `session::save`
- `sentinel::connect`, `sentinel::stream`
- `agentic::run`, `agentic::review`, `agentic::drift`
- `mapping_library::save`, `mapping_library::apply`
- `dlq::list`, `dlq::retry`
- `invariants::check`

Son 55 endpoints HTTP cuando el Tauri los expone, y 54 tools MCP cuando se exponen vía ese transporte. Pero la *lógica* vive una sola vez, en `platform/api`. ADR-001 formaliza esta regla: la API es canónica; ninguna otra capa reimplementa parcial ni "mejora" por dentro.

Esta decisión rompió el ciclo lógico que existía antes del rediseño (`platform/api` → `apps/cli` → `core`). Hoy `apps/cli` es un cliente más de la API, igual que Tauri.

5.2. `platform/dsl`

El DSL de hipótesis. La gramática actual acepta cadenas como:

```
IP -[Connect]-> Host -[Auth]-> User {k=3}
```

La gramática vive en `grammar/v1.rs`. Un trait `DslExtension` permite registrar nuevos tipos de entidad, predicados y operadores sin modificar el parser base. ADR-005 detalla el esquema de versionado.

5.3. platform/parsers

Hogar del trait `LogParser` con siete implementaciones: sysmon, sentinel, generic CSV, FortiAnalyzer, Cognito, IIS y un parser fallback. La decisión de qué parser usar se toma en `platform/api::format_regist`

5.4. platform/sources

Trait `LogSource` para fuentes en vivo: Sentinel, Defender XDR, Data Lake, y archivos locales. Es la contraparte "pull" de `platform/parsers` (que trabaja sobre bytes) y vive separada para poder reemplazarla sin tocar el parseo.

5.5. platform/canonical

Proyección OCSF v1.4 del modelo interno. Es la forma canónica que se exporta hacia SIEMs externos o hacia el IOC exporter. La dualidad "modelo interno rico / modelo externo canónico" permite evolucionar el interno sin cambiar el contrato público.

5.6. platform/vrl

Compilador de `FieldConfig` a VRL (Vector Remap Language). Se usa en el camino de ingesta cuando la fuente viene con un mapeo declarativo de campos. Contrato estable hacia clientes que comparten configuraciones (ver `platform/api::mapping_library`).

5.7. platform/constrained-decode

Guardrail para salidas de LLM. Impone una gramática a la generación (JSON strict, DSL de hipótesis, canonical query) para evitar que el modelo produzca texto inválido. Vive aquí porque es una superficie pública vista por el sistema agentic.

5.8. platform/siem

Streaming desde Sentinel y Elastic. Se extrajo de `apps/cli` para romper el ciclo que describe ADR-001. Hoy lo consumen tanto `platform/api` (para hacer ingest continuo desde el Tauri) como `apps/cli` (para ingesta batch desde terminal).

5.9. platform/local-llm

Trait `LocalLlm` con backends `mock` y `candle`. Permite al sistema agentic correr sin dependencias externas. Es superficie extensible: agregar un backend nuevo es implementar el trait, no tocar a los consumidores.

5.10. platform/mcp (TypeScript)

Servidor MCP con 54 tools organizadas en 10 dominios. No compila junto a los crates Rust: tiene su propio `package.json` y build (`npm run build`). Cada tool es un cliente HTTP liviano contra `platform/api`. ADR-004 formaliza el contrato `Tool<I, O>` y el versionado por tool.

5.11. Qué tienen en común

Los nueve crates Rust de `platform/` comparten tres rasgos:

- **Traits como contrato**: cada crate publica al menos un trait que define su extensibilidad (`LogParser`, `LogSource`, `DslExtension`, `LocalLlm`, `Tool` en el caso MCP).
- **DTOs versionados**: los tipos que cruzan fronteras (entre procesos o entre la API y el exterior) viven en `platform/api/src/dto/v1/` con compatibilidad semver.
- **Cero dependencia sobre aplicaciones**: un crate de plataforma nunca pide algo de `apps/`.

Esta uniformidad es lo que hace que el sistema se extienda sin regresiones: un nuevo parser se añade en `platform/parsers` sin que eso fuerce tocar ni el núcleo ni las aplicaciones.

Capítulo 6

Capa de aplicaciones

Las aplicaciones son la piel visible del sistema. Son tres: `apps/tauri` (GUI de escritorio), `apps/cli` (línea de comandos) y el servidor MCP TypeScript `platform/mcp`. Las tres dependen de `platform/api` como cliente; ninguna reimplementa lógica propia.

6.1. `apps/tauri`

El desktop app. Su propósito es presentar el grafo, correr hunts interactivos, mostrar scoring, y permitir anotación. Estructura interna:

- `src-tauri/src/commands/`: handlers de IPC Tauri, uno por dominio funcional (ingest, hunt, export, session, …). Cada comando es un **pass-through**: recibe el payload, llama al método correspondiente de `platform/api`, devuelve el resultado.
- `src-tauri/src/http_api.rs`: routers Axum que exponen los 55 endpoints HTTP. Sirven tanto al MCP server como a scripts externos. Estos routers no tienen lógica; son adaptadores de transporte.
- `src/`: frontend React. Cytoscape para el grafo, componentes por vista (Hunt, Explore, Datasets, Ingest, Notes, Export). Context API, sin estado global pesado.

La consigna aquí es **YAGNI agresivo**: ningún componente del Tauri debe anticipar variación que todavía no existe. Si dos comandos comparten boilerplate, se factoriza a un helper, no a un trait con una sola implementación.

6.2. `apps/cli`

El CLI es un consumidor in-proc de `platform/api`. No corre un servidor HTTP; crea una instancia de `GraphHunterApi`, llama operaciones y escribe a stdout en formato legible o JSON.

Tres usos principales:

- Ingesta batch de archivos (EVTX, CSV) a un workspace nuevo.
- Ejecutar un hunt desde un archivo de hipótesis y volcar resultados.
- Interactuar con `platform/siem` para trabajos programados de streaming (cron o scheduler externo).

El CLI tiene **cero** dependencia nueva sobre `core/` desde la Fase 2: todo pasa por la API. Esto dejó obsoleta a la antigua "ruta paralela" del CLI que bifurcaba la lógica de ingesta.

6.3. platform/mcp

El servidor MCP (Model Context Protocol) es un paquete TypeScript que expone 54 tools organizadas en 10 dominios, consumibles por clientes de IA como Claude Code o Cursor.

Arquitectura (post ADR-004):

- `src/lib/types.ts`: interfaz `Tool<I, O>` tipada, con `stability` explícita y `version`.
- `src/lib/registry.ts`: registro dinámico, agrupado por dominio.
- `src/tools/<dominio>/<tool>.ts`: una tool por archivo. Cada archivo expone un objeto que cumple el contrato `Tool<I, O>`.

Los 10 dominios son: `ingest`, `hunt`, `explore`, `export`, `session`, `agentic`, `dlq`, `invariants`, `mapping\_library`, `sentinel`.

Cada tool es un cliente HTTP contra `platform/api`. No hay lógica de negocio en el MCP; cambios en el ranking de hunts, por ejemplo, viven en `platform/api` una sola vez y se propagan automáticamente.

6.4. Por qué el MCP es plataforma, no aplicación

Aunque el paquete vive físicamente en `platform/mcp/` y nosotros lo tratamos como aplicación en el diagrama, la clasificación oficial es *plataforma*. La razón: el MCP define **un contrato público** visto por clientes externos (Claude, Cursor, custom agents). Cambios en su superficie afectan consumidores que no controlamos, igual que cambios en los 55 endpoints HTTP. Por eso lleva el rigor de plataforma: versionado por tool, tests contractuales, gate de estabilidad.

6.5. Convenciones de aplicaciones

Tres reglas operacionales para este nivel:

1. **Delgado**: si un comando Tauri o una tool MCP contiene más de, digamos, quince líneas de lógica real, hay que mover esa lógica a `platform/api` y volver a dejar la superficie delgada.
2. **Sin tests unitarios pesados**: smoke end-to-end sí; tests con mocks para cada handler, no. Si la lógica merece test unitario, pertenece a `platform/api` o al núcleo.
3. **Sin traits preventivos**: no se introducen traits en `apps/` salvo que haya dos implementaciones reales *hoy*. Esto contrasta con el estilo de `platform/`, y es deliberado.

Capítulo 7

Patrones transversales

Hay cinco patrones que se repiten en todo el código del proyecto. Cada uno resuelve un problema concreto; reconocerlos acelera la lectura de cualquier módulo nuevo.

7.1. Typestate

Codificar invariantes en el sistema de tipos para que estados inválidos no compilen. El ejemplo paradigmático fue `GmGraph` en `core/matcher-ffi`, eliminado junto con la frontera FFI el 2026-05-07. El patrón sigue siendo válido para cualquier máquina de estados futura del sistema; el código abajo se conserva como referencia:

```
pub struct GmGraph<S: State> {
    ptr: *mut c_void,
    _state: PhantomData<S>,
}

pub struct Building;
pub struct Finalized;

impl GmGraph<Building> {
    pub fn add_edge(&mut self, ...) -> Result<()> { ... }
    pub fn finalize(self) -> GmGraph<Finalized> { ... }
}

impl GmGraph<Finalized> {
    pub fn search(&self, ...) -> GmResults { ... }
}
```

`search` sólo existe en el estado `Finalized`; intentar buscar en un `GmGraph<Building>` es un error de compilación, no de runtime.

7.2. RAII sobre handles C

Nota 2026-05-07: la sección siguiente describe el patrón RAII sobre los handles C de `libgraphmatch`; ese motor fue eliminado y la frontera FFI ya no existe. Se conserva el texto como ejemplo del patrón aplicable a cualquier futuro FFI.

Todo puntero crudo que retorna el motor histórico `libgraphmatch` quedaba envuelto en una struct Rust que implementa `Drop` llamando a la función `gm\_*\_free` correspondiente. Esto garantiza:

- Liberación determinística en el scope donde se crea el handle.

- Liberación correcta ante `panic` (drop se ejecuta durante el unwind).
- Imposibilidad de "double free" — el tipo consume su contenido al liberar.

Aplica a `GmGraph`, `GmMatcher` y `GmResults`. Antes del rediseño, `GmMatcher` liberaba manualmente dentro de una función, lo que abría una ventana unsafe ante panic.

7.3. Canonical API

Una pieza de lógica se implementa **una sola vez**, en `platform/api`, y todos los transportes (HTTP, IPC, stdin, MCP) son adaptadores sobre esa misma lógica. Lo opuesto a la estructura anterior, donde Tauri, CLI y API reimplementaban cada uno partes del mismo flujo.

La regla operativa: si te encuentras copiando código de un crate de aplicación a otro, ese código debería estar viviendo en `platform/api`.

7.4. Model Gate

Separar la adquisición de un modelo (archivo ONNX) de su uso mediante una capa de validación entre ambos:

```
pub struct ModelGate { ... }

impl ModelGate {
    pub fn validate(path: &Path, expected: &ModelSpec)
        -> Result<ValidatedModel, GateError> { ... }
}

pub struct ValidatedModel { /* inference-ready */ }
```

Sólo `ValidatedModel` puede pasarse al inferencer. Si la validación falla (checksum, dimensiones, versión), el caller decide qué hacer: el patrón no fuerza un crash, pero sí impide pasar un modelo no validado accidentalmente.

Uso actual: `core/gnn/common/model_gate.rs` decide si el scorer GNN se habilita.

7.5. Registry de extensiones

Donde la plataforma necesita extensibilidad dinámica (sumar un parser, un entity type del DSL, una tool MCP) se usa un registry:

```
pub trait LogParser: Send + Sync {
    fn name(&self) -> &str;
    fn can_parse(&self, sample: &[u8]) -> bool;
    fn parse(&self, bytes: &[u8]) -> Result<Vec<Event>>;
}

pub struct ParserRegistry {
    parsers: Vec<Box<dyn LogParser>>,
}

impl ParserRegistry {
    pub fn register(&mut self, p: Box<dyn LogParser>) { ... }
    pub fn select(&self, sample: &[u8]) -> Option<&dyn LogParser>
      { ... }
}
```

Variantes del mismo patrón viven en `platform/dsl` (registry de extensiones de gramática), `platform/mcp` (registry de tools) y `platform/local-llm` (registry de backends).

El contrato es siempre: trait estable, registry concreto, discovery en tiempo de arranque. *No* hay autodescubrimiento por reflexión; todos los registros son explícitos en código.

7.6. Por qué estos cinco y no otros

Son los patrones que *cruzan capas*. Hay muchas otras convenciones locales (por ejemplo, el estilo de errores en `platform/api` usando `thiserror`) pero no afectan cómo se diseñan módulos nuevos en otras capas.

Estos cinco sí: cada uno tiene un costo (aumenta el código boilerplate o exige conocimiento previo) compensado por un beneficio estructural (imposibilita bugs, garantiza unicidad, desacopla extensibilidad). Cuando aparece un módulo nuevo que necesita invariantes en tipos, manejo de recursos externos, lógica pública, validación de artefactos o extensibilidad dinámica, se alcanza primero a uno de estos patrones antes de inventar otro.

Capítulo 8

Decisiones arquitectónicas

Siete ADRs documentan las decisiones que dieron forma al estado actual. El apéndice A contiene el texto completo; este capítulo las narra.

8.1. ADR-001 — Canonical API Layer

Problema. Antes del rediseño, `platform/api` dependía de `apps/cli` como librería para reusar lógica de streaming Sentinel; y `apps/cli` dependía del core directamente saltando la API. El resultado era un ciclo lógico (API → CLI → core) y una bifurcación: dos caminos de ingesta que iban a desincronizarse tarde o temprano.

Decisión. Extraer `sentinel\_streaming` a un crate nuevo `platform/siem` del que dependan ambos; hacer que `apps/cli` consuma `platform/api` in-proc como cliente más.

Consecuencia. `platform/api` es el único lugar con lógica; las aplicaciones son transportes. Cero ciclos.

8.2. ADR-002 — FFI Isolation (cerrada el 2026-05-07)

Problema. Un único `extern "C"` (bueno) pero con RAII parcial (peligroso): `GmMatcher` se liberaba manualmente. `GmGraph: Send + Sync` sin codificar la invariante "válido sólo tras finalize". Cinco funciones C declaradas sin uso desde Rust. Cero tests de equivalencia.

Decisión. Extraer el FFI a `core/matcher-ffi`. Aplicar typestate a `GmGraph`. RAII completo en `GmMatcher`. Añadir `gm_abi_version()` y chequeo en load-time. Property tests de equivalencia DFS ↔ SIMD.

Consecuencia (original). El aislamiento FFI pasó de convención a garantía codificada. Un binario con ABI mismatch crashea al arrancar en vez de devolver resultados sutilmente incorrectos.

Estado actual (2026-05-12). El motor C++ `libgraphmatch` fue eliminado el 2026-05-07 tras demostrar que el DFS puro-Rust lo superaba por 3-25× sobre cada workload medido. Las herramientas que esta ADR introdujo (typestate, RAII sobre handles foráneos, ABI version check) dejaron de tener un sujeto que defender. La intersección sorted-set AVX2 sobrevive como primitiva Rust pura en `core/graph-engine/src/simd\_rust.rs` con runtime dispatch. ADR se conserva como caso de estudio del patrón typestate aplicable a cualquier frontera FFI futura.

8.3. ADR-003 — GNN v2 Experimental

Problema. El modelo `gnn_v2.onnx` se referencia por string literal y *no* está entrenado sobre todos los datasets objetivo. Si un binario se envía con el modelo skeleton, el scorer devuelve probabilidades sin fallar visiblemente.

Decisión. Aislar GNN v2 detrás de `feature = "gnn-v2-experimental"`. Separar corpus export, training e inferencia en tres componentes con contratos claros. Añadir *model gate* que rechaza modelos no validados.

Consecuencia. Enviar un binario a producción con GNN v2 no-validado requiere flippear explícitamente el feature flag y pasar el gate. No es un accidente posible.

8.4. ADR-004 — MCP Tools Architecture

Problema. El servidor MCP original era un archivo de ~1670 líneas con 54 tools hardcoded. Cada tool llamaba al API por HTTP sin contrato tipado entre definición de tool y endpoint, así que mismatches se detectaban en producción.

Decisión. Modularizar: un archivo por tool, un registry por dominio, una interfaz `Tool<I, O>` tipada con `stability` y `version` explícitos. Tests contractuales contra `platform/api`.

Consecuencia. Cambiar una tool es un PR localizado. Deprecar una tool es una operación gobernable (marcar `deprecated`, dejar existir un ciclo, luego quitar).

8.5. ADR-005 — DSL Extensibility

Problema. El DSL vivía en `core/graph_hunter_core/src/dsl.rs` con entity types y relation types fijos. Clientes pedían `Container`, `ServiceAccount`, `Cluster`; predicados `in\_subnet`, `matches\_regex`; operadores temporales.

Decisión. Extraer a `platform/dsl`. Introducir `DslExtension` trait para registrar entity types, relation types y predicados custom. Versionar gramática (`v1`, `v2-experimental`). Preservar compatibilidad de hipótesis existentes.

Consecuencia. El DSL deja de ser "hardcode en el core" y pasa a ser una superficie extensible. Un cliente puede pedir una extensión sin bloquear a otros.

8.6. ADR-006 — WIP Absorption

Problema. Al arrancar el rediseño, `git status` tenía 44 archivos modificados y 14 archivos untracked: trabajo coherente de iteraciones previas (agentic slow-lane, DLQ, invariant checker) que no era código abandonado sino baseline no commiteado.

Decisión. Antes de tocar nada de la reorganización, absorber el WIP como *tres commits atómicos de baseline*. No descartar, no posponer: integrar para poder avanzar.

Consecuencia. El historial del rediseño arranca de una base coherente. Los cambios que siguen son estructurales, no mezcla de nuevas features + mover piezas.

8.7. ADR-007 — Ingestor agéntico en tres carriles

Problema. La ingesta vía parsers hardcodeados en el daemon es estática: un formato nuevo o un cambio aguas arriba obliga a release. El parser genérico cubre casos conocidos y falla silencioso en los demás. Ninguno aprende de su operación.

Decisión. Separar la ingesta en tres carriles (cap. 10): fast lane VRL compilado determinísticamente, slow lane LLM-as-compiler offline con gate humano, y learning lane que usa drift estructural como señal de re-review. El contrato entre carriles es el `FieldConfig` + mapping OCSF, ambos datos.

Consecuencia. El daemon no se recompila por formato nuevo; se publica un nuevo mapping y el runtime lo recoge. El LLM nunca toca datos en runtime: propone, humano aprueba, compilador emite, fast lane ejecuta. La superficie crece (`platform/vrl`, `platform/mapping-library`, `platform/local-llm`, tools MCP agénticas) pero la capa core sigue intacta — shearing-layers en acción.

8.8. Cómo encadenan

Las siete ADRs no son independientes. ADR-006 es condición necesaria para poder hacer ADR-001 (imposible reorganizar con trabajo suelto por arriba). ADR-001 habilita ADR-002 (mover el FFI a su propio crate requiere haber roto el ciclo). ADR-005 y ADR-004 son consecuencias de haber declarado la plataforma como superficie versionable. ADR-003 es la aplicación del patrón *model gate* (capítulo 7) al caso concreto del GNN. ADR-007 reusa ese mismo patrón para el LLM del slow lane: el compilador es opcional, remplazable, y sus fallas no afectan el fast lane.

Esto importa cuando se revisa una propuesta futura: si toca algo cubierto por una ADR, hay que actualizar la ADR, no saltarla.

Capítulo 9

Deudas y trabajo futuro

Este capítulo documenta lo que *no* está resuelto. Hay cuatro categorías: aristas indebidas en el grafo de dependencias, tests flaky, brechas de modelo y follow-ups del sistema agentic.

9.1. Dos aristas `core` → `platform` (con *guardrail* automatizado)

Como registra el capítulo 3, existen dos dependencias que van hacia arriba:

- `core/graph-engine` → `platform/sources`: el GNN bridge usa `AadEventBatch` definido en `platform/sources` para materializar el corpus de entrenamiento. Es un préstamo de tipo; se resuelve moviendo `AadEventBatch` a un crate más bajo (candidato: un crate nuevo `core/ingest-types`) o invirtiendo la dirección vía un trait.
- `core/graph-engine` → `platform/dsl`: el compilador de hipótesis y el runtime del grafo comparten tipos; separarlos completamente exige introducir un crate intermedio de tipos compartidos.

No son ciclos (el grafo sigue siendo DAG) pero violan la regla de capas. **A partir del ciclo 2026-04 ambas aristas están explícitamente *whitelisted* en `docs/arquitectura/layering-exceptions.yaml` y verificadas en cada PR por `scripts/check-layering.sh`.** Esto convierte la deuda en regresión visible: cualquier nueva arista `core` → `platform` no-whitelisted rompe CI. El indicador de éxito de largo plazo sigue siendo que `cargo metadata` reporte cero aristas hacia arriba; el indicador de corto plazo —cerrado— es que ninguna arista nueva pueda colarse silenciosamente.

9.2. Tests flaky conocidos

- `sentinel\_connector::tests::backoff\_retries` — depende de timing con `tokio::time::advance`, a veces falla en CI bajo carga. Mitigación actual: reintentar el job. Solución: rediseñar el test para usar relojes virtuales sin depender de wall-clock.
- Smoke tests de ingesta que abren archivos grandes en paralelo saturan I/O en runners GitHub y producen timeouts esporádicos. Mitigación: serializar la fase de ingesta en CI.

Ninguno se ha visto fallar localmente de forma reproducible. No bloquean releases pero contaminan el ruido de CI.

9.3. GNN v2 sin modelo entrenado

ADR-003 previene que un binario acepte el modelo skeleton, pero no resuelve la brecha subyacente: el entrenamiento de GNN v2 está hecho sobre Sentinel/AAD y no se ha validado contra datasets Fortigate o IIS. Dos consecuencias:

- La componente `gnn\_threat` del scoring compuesto no aporta señal útil cuando se alimenta con logs fuera del dominio entrenado. En esos casos el gate rechaza la carga y el scoring se reduce a cuatro componentes. Es el comportamiento correcto, pero es menos discriminante.

- No hay *corpus export* operativo para FortiAnalyzer; el trabajo necesario incluye estructurar el pipeline de exportación + entrenamiento para esa familia de logs.

9.4. Follow-ups agentic slow lane

El sistema agentic (integrado en Fase 2 vía ADR-006) tiene tres piezas pendientes:

- **Backend local-llm real**: actualmente el backend `candle` está cableado pero el modelo de bajo recurso no converge con calidad suficiente para el *review loop*. Falta decidir entre (a) afinar el modelo actual, (b) integrar un backend remoto con API key rotada, o (c) ambos con fallback.

- **Drift detection feedback loop**: la operación `agentic::drift` detecta schema drift pero no propaga la corrección al `mapping\_library`. Falta cablear esa vuelta para que un drift detectado genere una propuesta de re-mapping revisable.

- **DLQ triage UI**: la DLQ es funcional por API pero no tiene vista en el Tauri. Consecuencia: un operador ve "hay N eventos fallidos" pero no puede triagearlos sin `curl`.

9.5. Deudas cerradas en el ciclo 2026-04

Se dejan registradas aquí, con puntero a la evidencia, para que la próxima revisión del capítulo no tenga que re-descubrirlas:

- **FFI sin anotaciones** `SAFETY`: cerrada en el ciclo 2026-04 anotando todos los bloques `unsafe` de `core/matcher-ffi/src/lib.rs` con `// SAFETY: ...` y habilitando el lint `missing-safety-doc`. Posteriormente (2026-05-07) la frontera FFI completa fue eliminada junto con `libgraphmatch`; ya no hay bloques `unsafe` en el crate residual, así que la deuda quedó *materialmente cerrada por sustracción*.

- **DSL v1 sin semántica formal**: publicada en `docs/spec/dsl-v1-semantics.md`. Cubre definición de *match* (paths vs walks), semántica de `WITHIN`, `\{k=N\}`, `HAVING count(distinct)`, y los predicados `\textasciitilde` , `in`, `not in`. El roundtrip proptest cubre ahora bindings, HAVING y WITHIN que antes se excluían.

- **Matcher 3-way equivalence**: cerrada con el proptest histórico `core/matcher-ffi/tests/equivalence\_pr` que comparaba iterativo vs smart vs SIMD a 256 casos con resultados idénticos para scorer finalizado. Tras la eliminación del path C++ (2026-05-07) sólo quedan los caminos *iterativo* y *smart*; la equivalencia AVX2 vs escalar se valida en los 15 tests de `core/graph-engine/src/simd\_rust.rs` y la paridad LFTJ *naive = LFTJ = LFTJ+SIMD* en `core/graph-engine/src/lftj.rs`.

- **Scoring sin** `ScoreBreakdown` **documentado**: publicado en `docs/spec/scoring-v1.md`. Perfil por defecto con `w\_gnn = 0`, breakdown por path expuesto en `analytics::HuntResult::anomaly\_bre` tests de reproducibilidad bit-for-bit (`f64::to\_bits`) y direccionalidad.

- **Escenarios demo no reproducibles**: `scripts/demo.sh` + snapshot tests en `platform/api/tests/demo\_sc` cubren los tres fixtures (Sysmon, Sentinel, CSV) con tolerancia ±10 % sobre el número de paths.

- **Benchmarks limitados a 2 suites**: agregadas `ingest\_throughput`, `matching\_hops` y `scoring\_path`. `scripts/bench-report.sh` convierte `target/criterion/*/estimates.json` en `docs/paper/data/bench-table.md`.
- **Feature flags sin tabla canónica**: publicada en `docs/spec/features.md` con clasificación stable/experimental/future sobre 14 flags.

9.6. Formato del libro

Este capítulo se actualiza en cada ciclo de trabajo. Si una deuda se resuelve, se elimina de aquí y se anota en la ADR correspondiente bajo "consecuencias". Si aparece una nueva, se documenta aquí antes de que alguien la descubra por sorpresa durante un incidente.

La regla editorial: las deudas se enumeran con nombre de módulo, efecto observable y criterio de resolución. No se usan frases vagas ("mejorar"); se usan criterios verificables ("cero aristas hacia arriba en `cargo metadata`").

Capítulo 10

Ingestor agéntico (tres carriles)

10.1. Motivación

La ingesta heurística original —un parser por formato, compilado en el binario— cubre los formatos conocidos y se acaba: cada formato nuevo requiere release del daemon, y cada cambio de schema aguas arriba (Sentinel agrega un campo, FortiOS cambia un separador) rompe el parser en producción.

El rediseño divide la ingesta en *tres carriles* independientes, cada uno con un perfil de latencia, determinismo y superficie de cambio distinto:

1. **Fast lane (VRL compilado)**: ruta caliente, determinística, sin LLM, sin red. Un `FieldConfig` ya validado compila a un programa VRL; el daemon lo ejecuta para cada batch. Es el *único* carril que toca la ingesta productiva.

2. **Slow lane (LLM-as-compiler, offline)**: cuando llega un formato nuevo o cambia uno conocido, un LLM analiza una muestra y propone un `FieldConfig` + mapping OCSF. El output pasa por revisión humana en la UI agéntica; al aprobarse, el compilador VRL emite el programa para el fast lane. El LLM *no* toca datos en runtime.

3. **Learning lane (drift-as-signal)**: el fast lane mide, a costo casi cero, métricas estructurales por batch (distribución de tipos, cardinalidad de campos, ratio de DLQ). Cuando un batch se desvía del perfil esperado más allá de un umbral, se dispara re-revisión en el slow lane. El fast lane no se para: sigue ingestando con el mapping actual mientras la propuesta madura.

10.2. Flujo end-to-end

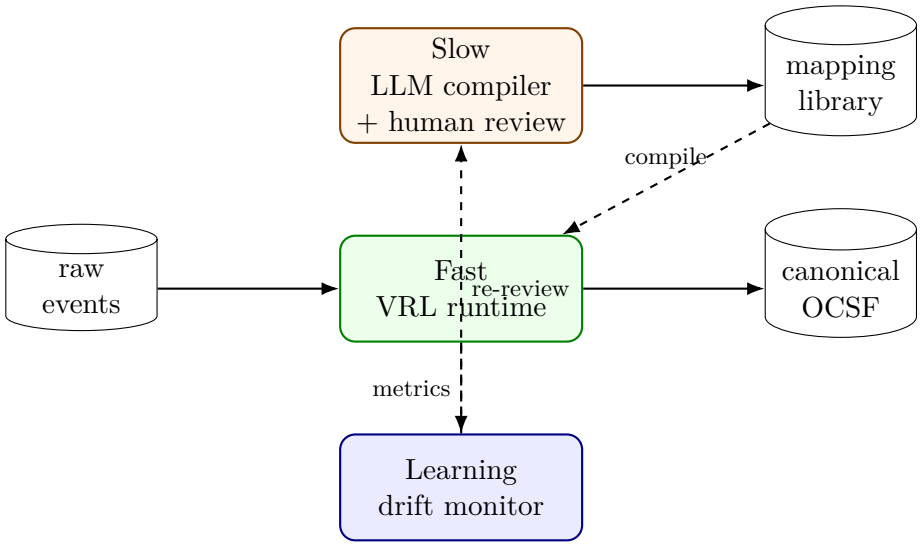


Figura 10.1: Tres carriles. Flechas sólidas = ruta de datos en runtime. Flechas punteadas = feedback y compilación offline.

El contrato entre carriles es el `FieldConfig` + el mapping OCSF, ambos datos. Ningún carril llama a otro por función: el slow lane emite un artefacto que el fast lane recoge, el learning lane emite una señal que el slow lane consume.

10.3. Fast lane: VRL compilado

El crate `platform/vrl` toma un `FieldConfig` y emite un programa VRL textual. La compilación es pura:

$$\text{VRL}(f) = \text{compile}(f) \qquad \text{mapping_hash}(f) = \text{sha256}(\text{VRL}(f))$$

El `mapping_hash` identifica al programa y se estampa en cada evento emitido (campo `provenance.mapping_h` cerrando el circuito entre un evento canonical y el `FieldConfig` que lo produjo. El runtime es un intérprete VRL embebido; el reemplazo por la crate `vrl` upstream es follow-up de performance, no de semántica.

Consecuencias:

- Cambiar un mapping no requiere release del daemon; sólo recompilar el VRL y reapuntar.
- Un batch con el mismo `mapping_hash` es reproducible byte-a-byte: ingesta determinística.
- El fast lane nunca llama a un LLM. Seguridad y latencia predecibles.

10.4. Slow lane: LLM-as-compiler offline

El LLM se usa como *compilador* de ejemplos a `FieldConfig`. El prompt recibe:

- Muestra del formato crudo (20-100 líneas).
- Catálogo de tipos de entidad y relación disponibles.
- Schema OCSF target.

Y emite un `FieldConfig` propuesto + mapping OCSF. Esta propuesta entra a la cola de review agéntica (`POST /agentic_review_list`). Un humano aprueba, edita o rechaza. Sólo tras aprobación el compilador VRL genera el programa de fast lane.

Propiedades:

- El LLM jamás procesa eventos de producción. Opera sobre muestras explícitas.
- El output es datos (`FieldConfig`), no código. El código lo genera el compilador determinístico.
- El humano es el último filtro: mismas garantías de un mapping curado a mano, con el boilerplate automatizado.
- Backend LLM intercambiable (`platform/local-llm`): local GGUF o API remota según configuración.

10.5. Learning lane: drift-as-signal

Cada batch en fast lane emite métricas baratas:

- Histograma de entity types producidos.
- Cardinalidad de cada campo canonical.
- Ratio de eventos que fueron a DLQ.
- Distribución de timestamps (gap, skew).

El monitor de drift (`platform/api/src/operations/agentic_drift.rs`) compara estas métricas contra el perfil esperado del mapping. Un Kolmogorov-Smirnov por campo, con ventana móvil. Si el drift supera el umbral:

1. Se abre una propuesta de re-review apuntando al mapping actual.
2. El batch *sigue* ingestando con el mapping viejo — ingest-or-drop, no ingest-or-stall.
3. El slow lane retoma: el LLM ve el batch anómalo + el mapping actual, propone delta.
4. Humano aprueba, nuevo `mapping_hash`, el fast lane recoge la nueva versión en el siguiente batch.

El drift no es un error, es una señal. El sistema nunca deja de ingestar mientras aprende.

10.6. Estabilidad de cada carril

	Fast	Slow	Learning
Determinismo	✓	parcial (hash estable del output)	✓
Latencia	runtime (ms)	offline (minutos)	runtime (por batch)
Depende de LLM	nunca	sí	no
Puede fallar caliente	sí (datos)	n/a	n/a
Gating humano	ninguno	mandatorio	ninguno

Tabla 10.1: Perfil de cada carril. El único carril con latencia en el camino crítico es el fast lane; sólo el slow requiere LLM y humano.

10.7. Relación con el resto de la arquitectura

El ingestor agéntico vive íntegramente en la *capa platform* (ver cap. 5). El core no cambia: sigue recibiendo `Event` canonical como siempre. Los cambios están concentrados en `platform/vrl`, `platform/mapping-library`, `platform/local-llm` y `platform/api/src/operations/agentic*.rs`.

Esto es consistente con shearing-layers: un cambio de política de ingesta (pasar de parser-compilado a VRL-compilado-desde-LLM) se absorbe en la capa de cambio medio sin tocar el núcleo (motor de grafo y matcher SIMD).

10.8. Qué queda fuera de este libro

- La semántica concreta del lenguaje VRL emitido y el mapeo canonical OCSF están en el libro de Especificación (ver cap. *Formatos de log* y cap. *MCP tools*, dominio `agentic`).
- La evaluación cuantitativa (latencia del fast lane, accuracy del LLM, tasa de drift real) vive en el Paper.

Capítulo 11

Estado de implementación

Este capítulo es el *cut* de lo que está implementado vs lo que está planificado para el ciclo en curso. No es exhaustivo —la ADR y el código son la fuente de verdad— pero resuelve la pregunta operativa del lector: "si abro `core` ahora, ¿qué voy a encontrar funcional y qué voy a encontrar en *feature flag*?"

11.1. Matriz *stable / experimental / future*

Todas las *features* Cargo están catalogadas en `docs/spec/features.md`; aquí se resume la clasificación agrupada por subsistema.

Subsistema	Estado	Feature gate
Grafo y matcher DFS	stable	(default)
Intersección AVX2 (`simd\_rust.rs`)	stable	(default, runtime dispatch)
Scoring v1 (4 componentes)	stable	(default)
GeoIP offline	stable	`geoip`
FFI typestate/RAII (ADR-002, cerrada)	removido 2026-05-07	—
Canonical OCSF v1.4	stable	(default)
Parsers Sysmon/Sentinel	stable	(default)
Parser FortiAnalyzer	stable	(default)
DSL v1 (`WITHIN`, `HAVING`)	stable	(default)
DLQ + reingest	stable	(default)
Ingestor agéntico 3-carriles	stable	(default)
GNN v2 (ADR-003, gate activo)	experimental	`gnn-v2-experimental`
ONNX Runtime backend	experimental	`ml-scoring`
DirectML backend (Win GPU)	experimental	`directml`
Backend local-llm (candle)	experimental	`candle`
DSL v2 (registries estrictos)	experimental	(en compilación)
VRL constrained-decode (xgrammar CFG)	experimental	`xgrammar`
Drift → mapping_library loop	future	(sin flag aún)
DLQ triage UI en Tauri	future	(sin flag, sólo API)

11.2. Invariantes del *default build*

Con `cargo build --workspace` (sin *features*) se garantiza:

1. Ningún símbolo de ORT ni de un modelo GNN queda linked. Verificable con `cargo tree -p graph\_hunter\_gnn --no-default-features`.

2. Ninguna llamada de red en *startup*.
3. El *scoring* compuesto es determinista: tests `scoring\_v1\_is\_bit\_for\_bit\_reproducible` y `scoring\_v1\_default\_profile\_ignores\_injected\_gnn\_scores` pinnean esta propiedad.
4. `scripts/check-layering.sh` verifica que no existan aristas `core` → `platform` no-whitelisted. Las dos aristas legítimas están registradas en `docs/arquitectura/layering-exceptions.txt`.

11.3. Puntos de entrada para auditores

- `docs/spec/dsl-v1-semantics.md`: semántica formal del lenguaje de hipótesis.
- `docs/spec/scoring-v1.md`: fórmulas y perfil por defecto del *scoring* explicable.
- `docs/spec/features.md`: tabla canónica de *feature flags* (14 entradas).
- `docs/paper/data/bench-table.md`: generada desde `target/criterion/*/estimates.json` por `scripts/bench-report.sh`.
- `scripts/demo.sh`: tres escenarios reproducibles *end-to-end* (Sysmon office-spawn, Sentinel lateral-movement, CSV exfil-dns).

11.4. Política de cambios al capítulo

Cada vez que un *flag* pasa de `experimental` a `stable` (o entra una nueva categoría `future`), se actualiza esta tabla **en el mismo PR** que modifica el *Cargo.toml*. Si la tabla y el código divergen, el código gana —pero la divergencia cuenta como bug de documentación y bloquea el merge.

Apéndice A

ADRs: registros de decisión

Este apéndice recoge las siete ADRs (Architectural Decision Records) que fundamentan el diseño actual. El formato es el mismo para todas: contexto breve, decisión, consecuencias. El capítulo 8 del cuerpo principal narra cómo encajan entre sí; aquí quedan en su forma de referencia.

Fuente primaria: `docs/_archive/adrs/001..006.md` en el repositorio. Si hay discrepancia entre este apéndice y esos archivos, prevalece el archivo.

ADR-001 — Canonical API Layer

Fecha: 2026-04-23. **Estado**: aprobada e implementada (Fase 2).

Contexto. `platform/api` dependía de `apps/cli` como librería para reusar `sentinel_streaming`, creando un ciclo lógico API → CLI → core. `apps/cli` además saltaba la API yendo directo al core, bifurcando la ruta de ingesta.

Decisión. (D1) Extraer `sentinel_streaming` a un crate nuevo `platform/siem` del que dependen API y CLI. (D2) `apps/cli` pasa a ser cliente in-proc de `platform/api` (descartada la variante HTTP). (D3) DTOs en `platform/api/src/dto/v1/` versionados con semver.

Consecuencias. Cero ciclos en `cargo metadata`. Una única lógica, varios transportes. Extender la API requiere versionar DTOs y dejar el endpoint viejo funcional durante un ciclo.

ADR-002 — FFI Isolation

Fecha: 2026-04-23. **Estado**: aprobada e implementada (Fase 3).

Contexto. Un único `extern "C"` (bueno), pero RAII parcial, invariante "válido tras `finalize()`" no codificada, cinco funciones C sin uso desde Rust, cero tests de equivalencia, build script sin chequeo de ABI.

Decisión. (D1) Extraer la fachada FFI a `core/matcher-ffi`. (D2) Typestate para `GmGraph` con `Building`/`Finalized`. (D3) RAII completo en `GmMatcher`. (D4) `gm_abi_version()` con check en load-time; mismatch ⇒ crash temprano. (D5) Property tests de equivalencia DFS ↔ SIMD.

Consecuencias. El aislamiento FFI pasa de convención a garantía de tipo. Imposible ejecutar un hunt sobre un grafo no finalizado. Imposible shippear con un objeto compartido de ABI distinta sin que el binario crashee al arranque.

Estado (2026-05-07): `libgraphmatch` fue eliminado tras demostrarse que el DFS puro-Rust lo supera en 3-25×. Los wrappers typestate y RAII descritos arriba ya no existen; `core/matcher-ffi` es ahora un *thin forwarder* Rust-puro de ∼30 líneas. Esta ADR se conserva como contexto histórico.

ADR-003 — GNN v2 Experimental

Fecha: 2026-04-23. **Estado**: aprobada; corpus export y gate implementados; modelo no entrenado para Fortigate.

Contexto. `gnn_v2.onnx` referenciado en producción por string literal, no entrenado sobre todos los datasets objetivo. Binario enviado con skeleton produce probabilidades sin fallar visiblemente.

Decisión. (D1) Tres componentes separados: `core/gnn/v1` estable, `core/gnn/v2` detrás de `feature = "gnn-v2-experimental"`, `core/gnn/common` con el gate. (D2) `ModelGate::validate` rechaza archivos no firmados o con dimensiones incorrectas. (D3) Corpus export (`platform/sources/aad_training` separado del inferencer.

Consecuencias. Enviar un binario con GNN v2 no validado requiere (a) flippear el feature flag y (b) pasar el gate. Accidentalmente imposible. Extender a Fortigate exige nuevo corpus export + entrenamiento.

ADR-004 — MCP Tools Architecture

Fecha: 2026-04-23. **Estado**: aprobada e implementada (Fase 4).

Contexto. `graph-hunter-mcp/src/index.ts` era un monolito de ~1670 líneas con 54 tools hardcoded, sin contrato tipado entre definición de tool y endpoint HTTP.

Decisión. (D1) Interfaz `Tool<I, O>` tipada con `stability` y `version`. (D2) Registry modular por dominio (10 dominios). (D3) Un archivo por tool. (D4) Tests contractuales contra `platform/api`. (D5) El paquete vive en `platform/mcp/`; el paquete legacy `graph-hunter-mcp/` queda un ciclo más por compat.

Consecuencias. Cambiar una tool es un PR localizado. Deprecar una tool es una operación gobernable. Mismatches de contrato se ven en tests, no en producción.

ADR-005 — DSL Extensibility

Fecha: 2026-04-23. **Estado**: aprobada e implementada (Fase 3).

Contexto. El DSL vivía en `core/graph_hunter_core/src/dsl.rs`, con entity types y relation types hardcoded. Clientes pedían extensiones (`Container`, `ServiceAccount`, predicados temporales).

Decisión. (D1) Extraer a `platform/dsl`. (D2) Trait `DslExtension` para registrar entity/relation/predicate. (D3) Versionar gramática: `v1` estable, `v2-experimental` detrás de feature. (D4) Preservar compatibilidad de hipótesis escritas contra v1.

Consecuencias. El DSL deja de ser código del núcleo y pasa a ser superficie extensible. Un cliente puede extender sin bloquear a otros. Versiones futuras viven en paralelo hasta promoción.

ADR-006 — WIP Absorption

Fecha: 2026-04-23. **Estado**: aprobada e implementada (Fase 2, commit 0).

Contexto. `git status` al arrancar Fase 0: 44 archivos modificados + 14 untracked. No era código abandonado: iteraciones M4 (agentic slow-lane), M5 (DLQ), M6 (invariant checker) coherentes con el rediseño. Ignorarlas habría perdido trabajo; separarlas antes de reorganizar habría diluido la señal de cada commit.

Decisión. Tres commits atómicos *antes* de la reorganización: (1) M4 agentic + local-llm, (2) M5 DLQ, (3) M6 invariants. Cada commit incluye código + tests + DTOs asociados.

Consecuencias. El rediseño arranca desde una base coherente. Los commits siguientes son puramente estructurales, no mezcla de nuevas features con cambios arquitectónicos.

ADR-007 — Ingestor schema-agnóstico en tres carriles

Fecha: 2026-04-24. **Estado**: aprobada, en implementación (fast lane en `platform/vrl`; slow/-learning lanes en `platform/api/src/operations/agentic*.rs`).

Contexto. La ingesta heurística acopla un parser por formato en el binario del daemon. Cada formato nuevo o cambio aguas arriba (campo agregado en Sentinel, separador nuevo en FortiOS) obliga a release para que el daemon lo procese. El parser generic cubre casos conocidos y falla silencioso en los demás. Resultado: un SOC con un formato no reconocido se queda sin ingesta utilizable, o pierde fidelidad con el parser generic. No hay camino incremental: ninguno de los parsers aprende de su propia operación.

Decisión.

1. **Fast lane**: separar la ingesta caliente en un programa VRL compilado determinísticamente desde un `FieldConfig`. Cero LLM en el camino crítico.
2. **Slow lane**: usar un LLM *offline* como compilador de muestras a `FieldConfig`. El output del LLM es datos, no código. Todo output pasa por revisión humana antes de entrar a producción.
3. **Learning lane**: el fast lane emite métricas estructurales por batch; un monitor de drift dispara re-review en el slow lane sin parar la ingesta.
4. **Mapping library**: `platform/mapping-library` persiste los mappings aprobados con `mapping_hash = sha256(VRL)` como identidad estable y provenance-stamp por evento ingestado.
5. **Target canonical**: los mappings emiten OCSF v1.4 como shape canonical. El parser no decide qué es una entidad; el FieldConfig-a-OCSF sí.

Consecuencias.

- La frontera humano/máquina queda bien puesta: LLM propone, humano aprueba, compilador emite, runtime ejecuta. Cada paso es trazable.
- El daemon no necesita release por formato nuevo: se publica un nuevo `FieldConfig` y el runtime lo recoge.
- La determinística del fast lane se preserva: `mapping_hash` estable, reproducible byte-a-byte.
- Superficie nueva: tools MCP (`parser_generate`, `publish_mapping`, `agentic_review_*`, `schema_drift_detect`, `mapping_regression_test`, `fetch_shared_mappings`) que juntas materializan el carril agéntico.
- Dependencia operativa nueva (backend LLM). Mitigada por intercambiabilidad vía `platform/local-llm` y por que el carril es offline — caída del LLM no afecta ingesta viva.

Alternativas consideradas.

- **LLM online en el camino crítico**: descartado por latencia, costo y superficie de falla. El LLM puede parsear mal un batch productivo; el humano, no. Y el costo por evento ingerido sería prohibitivo.
- **Parser genérico mejorado**: descartado porque hereda la misma falla — no aprende de su propia ejecución ni se adapta a cambios aguas arriba.
- **Schema-registry estático aportado por usuario**: factible pero exige que el usuario escriba FieldConfig a mano. El slow lane automatiza ese paso sin perder el gate humano.

Apéndice B

Métricas del código

Snapshot al **2026-04-24** (post-cierre de Fase 4). Las métricas se reproducen con los comandos indicados al pie de cada tabla y se verifican en CI.

B.1. Líneas por crate

Capa	Crate	LOC
core	`core/graph-engine`	29 347
core	`core/matcher-ffi`	30
core	`core/gnn`	1 399
platform	`platform/api`	15 408
platform	`platform/canonical`	1 001
platform	`platform/constrained-decode`	443
platform	`platform/dsl`	1 716
platform	`platform/local-llm`	297
platform	`platform/parsers`	2 658
platform	`platform/siem`	744
platform	`platform/sources`	4 266
platform	`platform/vrl`	707
apps	`apps/cli`	610
apps	`apps/tauri/src-tauri`	5 117
Total Rust		**72 184**
TypeScript	`platform/mcp/src/`	1 975

Tabla B.1: LOC por crate/paquete (incluye código fuente, no tests ni benches fuera de `src/`). Nota 2026-05-07: la columna C++ con ∼12 787 líneas en `libgraphmatch/` fue eliminada tras la migración al DFS puro-Rust, y `core/matcher-ffi` cayó de 829 a ≈30 líneas al quedar como *thin forwarder*.

Reproducción: `find <crate>/src -name "*.rs" | xargs wc -l`.

B.2. Tests por crate

Crate	Archivos en `tests/`
`core/graph-engine`	22
`core/matcher-ffi`	0
`platform/api`	12
`platform/canonical`	1
`platform/constrained-decode`	2
`platform/dsl`	1
`platform/vrl`	1

Tabla B.2: Archivos de test de integración por crate. Tests unitarios embebidos en `src/` (`\#[cfg(test)] mod tests`) no están contabilizados aquí.

Reproducción: `find <crate>/tests -name "*.rs" | wc -l`.

B.3. Propiedades formales con property-testing

- Equivalencia AVX2 ↔ escalar de `intersect\_sorted\_u32\_count` en los 15 unit tests de `core/graph-engine/src/simd\_rust.rs`; paridad triple *naive* = *LFTJ* = *LFTJ+SIMD* sobre conteo de triángulos en `core/graph-engine/src/lftj.rs`. El proptest histórico `core/matcher-ffi/tests/equ` fue eliminado el 2026-05-07 junto con la frontera FFI.
- Roundtrip DSL parse → render → parse en `platform/dsl/tests/roundtrip.rs` (proptest).
- Invariantes de anomaly scoring en `core/graph-engine/tests/anomaly_proptest.rs`.

B.4. Ciclos arquitectónicos

Cero ciclos en `cargo metadata`. Se verifica con:

```
cargo metadata --format-version 1 --no-deps | \
  jq '[.packages[] | {name, deps: [.dependencies[].name]}]' | \
  # construir DAG
  # detectar SCCs (Tarjan)
```

El CI ejecuta esta verificación en cada push a main.

B.5. Aristas entre capas

Dirección	Estado	Conteo
`apps` → `platform`	Correcto	3
`platform` → `core`	Correcto	11
`core` → `platform`	**Deuda**	2
`apps` → `core`	Correcto (sólo tipos)	2
`platform` → `apps`	Prohibido	0
`core` → `apps`	Prohibido	0

Tabla B.3: Aristas entre capas del grafo de dependencias. Las dos de "deuda" están descritas en el capítulo 9.

B.6. Benchmarks definidos

- `core/graph-engine/benches/hunt_latency.rs`: latencia de `run_hunt` sobre grafos de tamaño creciente.
- `core/graph-engine/benches/dedup_throughput.rs`: throughput del interner bajo carga concurrente.

Los resultados concretos no se fijan en este documento — varían con hardware. CI compara contra baseline histórica y bloquea regresiones $> 15\,\%$.

B.7. Cambios respecto al pre-rediseño

Para contextualizar la evolución:

- **Ciclos**: 1 (API $\rightarrow$ CLI $\rightarrow$ core) $\rightarrow$ 0.
- **Aristas indebidas**: 4 $\rightarrow$ 2.
- **Crates**: 4 $\rightarrow$ 14 (la separación hizo explícitas fronteras que estaban implícitas).
- **LOC en el core**: no se comparan 1:1 porque el core antiguo agregaba piezas que ahora viven en plataforma; la comparación útil es "tamaño del algoritmo puro tras la limpieza" contra "tamaño del core de plataforma mezclada".